

دليل تسليم وتشغيل المشروع

F1 Car Vision Intelligence System

موجه لمطور يعرف أساسيات Python ويريد تشغيل المشروع وفهمه وتعديله من جهازه خطوة بخطوة.
هذا المستند

GitHub	github.com/MohammedHssan11/f1-car-vision-intelligence-system
Main commit	تحسين الاستقرار وتشغيل الفيديو داخل المتصفح - 3b0e956
Default URL	http://127.0.0.1:8010
Python	3.11 or newer

مهم قبل البدء

الكبيرة داخل /models لا ترفع إلى GitHub. يجب أن ترسل له هذه الملفات من جهازك بشكل منفصل قبل تشغيل video pipeline. المستودع يحتوي الكود والـ sample video، لكن ملفات الذكاء الاصطناعي

1. المشروع يعمل إيه؟

فيديو سباقات Formula-style. يأخذ فيديو سباق أو صورة سيارة ويخرج تحليل مرئي ومنظم، بدون telemetry رسمي من السيارة. هو نظام Computer Vision لتحليل يراقب الضرر، ويستنتج أحداثا مثل impact محتمل أو overtake heuristic. ثم يصدر فيديو MP4 عليه annotations وصفحة نتائج. في وضع الفيديو: يكتشف السيارات، يعطي كل سيارة ID ثابت قدر الإمكان، يتعرف على الفريق من ألوان السيارة، في وضع الصورة: يكتشف أنواع ضرر السيارة في صورة واحدة ويحفظ نسخة annotated image.

النماذج الأساسية

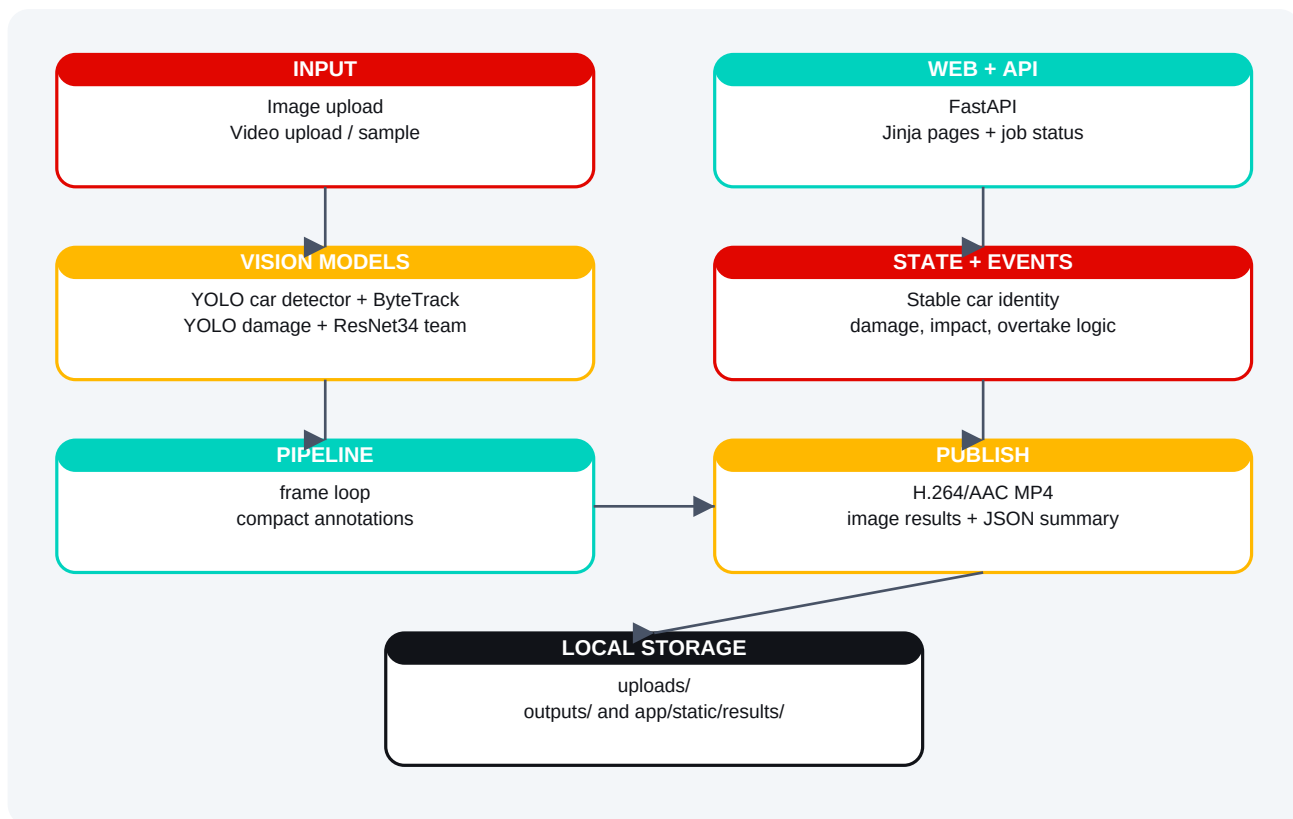
YOLOv8 car detector	يكتشف سيارة السباق داخل كل frame.
ByteTrack	يربط اكتشاف السيارة بين الفريمات ويعطي track ID.
YOLOv8 damage detector	يكتشف dent, scratch, tire flat وباقي فئات CarDD.
FastAI ResNet34	يتعرف على الفريق من livery السيارة.
OpenCV + ffmpeg	قراءة الفيديو، رسم overlays، ثم إخراج H.264/AAC مناسب للمتصفح.

ملاحظات الدقة

- السرعة بوحدة pixels/second وليست km/h، لأنها من صورة broadcast وليست GPS أو telemetry.
- يجب مراجعتها مع الفيديو، وليست قرارا رسميا للسباق heuristic نتائج collision و overtake.
- damage detector يحتاج لقطة واضحة؛ الضرر الصغير أو السيارة البعيدة قد لا يظهر.

2. Architecture المشروع

monolithic: FastAPI يستقبل الطلب، services تستدعي منطق الرؤية، وملفات tracking_memory تحفظ حالة السيارات لكل فيديو. الصفحة التالية تلخص مسار البيانات. التطبيق



النتائج لا تستخدم Database. uploads والـ outputs محفوظة محليا، وحالة السيارات موجودة في الذاكرة أثناء تنفيذ job فقط.

3. قبل أن يبدأ - ما الذي يجب أن يستلمه منك؟

يحتوي المحتويات التالية بنفس أسماء الملفات والمسارات. لا تغير الاسم لأن التطبيق يتحقق من hash لكل model قبل تحميله. أرسل له رابط GitHub، ثم ملف ZIP أو Google Drive

```
models/  
  best_f1_detect.pt  
  best_carDD.pt  
  f1_team_classifier.pkl
```

الحل هو إرسال نفس النسخ الموجودة عندك، أو تحديث hash المقابل في app/config.py بعد مراجعة النموذج الجديد. الملفات ليست في GitHub لأن gitignore يتجاهل *.pt و *.pkl. إذا كانت نسخة model مختلفة، سيظهر ModelIntegrityError. هذه

ما الذي يأتي من GitHub تلقائياً؟

- كل ملفات Python، docs، tests، Docker configuration، HTML، وملف tracker/tracker2.mp4 التجريبي.
- 3 صور benchmark صغيرة داخل ./splited_dataset/val/valid_images.
- لا ترسل outputs و uploads / الحالية؛ المشروع ينشئها تلقائياً.

متطلبات الجهاز

- Windows PowerShell الخطوات أدناه مكتوبة لـ Windows 10/11 أو Linux/macOS.
- اختياري لكنه أسرع جداً للفيديو GPU NVIDIA. أو أحدث Python 3.11.
- يثبت تلقائياً Docker image. مطلوب لإخراج فيديو يعمل داخل المتصفح ffmpeg.
- أو أكثر عند تحميل النماذج الثلاثة GB يفضل 16 RAM.

4. تشغيل المشروع من Windows PowerShell - GitHub

الأوامر بالترتيب. لا تفتح pipeline_result.html مباشرة؛ هذه Jinja template ويجب فتح التطبيق من المتصفح بعد تشغيل FastAPI. افتح PowerShell واكتب

```
git clone https://github.com/MohammedHssan11/f1-car-vision-intelligence-system.git  
cd f1-car-vision-intelligence-system  
  
py -3.11 -m venv .venv  
.\.venv\Scripts\Activate.ps1  
python -m pip install --upgrade pip  
pip install -r requirements.txt
```

الذي أرسلته له داخل folder المشروع. يجب أن يصبح المسار مثلاً: f1-car-vision-intelligence-system\models\best_f1_detect.pt بعد ذلك انسخ مجلد models

```
ffmpeg -version  
py -3.11 -m uvicorn app.main:app --host 127.0.0.1 --port 8010
```

ffmpeg غير موجود، ثبته ثم افتح PowerShell من جديد. يمكن تثبيته بـ winget install Gyan.FFmpeg، أو من الموقع الرسمي الخاص به حسب نظامه. لو ظهر أن

افتح بعدها في المتصفح

```
http://127.0.0.1:8010/  
http://127.0.0.1:8010/page/damage  
http://127.0.0.1:8010/page/pipeline  
http://127.0.0.1:8010/docs
```

يكون مستخدما من مشروع FastAPI آخر. لو 8010 مشغول، غير --port إلى 8011 وغير ALLOWED_ORIGINS في env. عند الحاجة. اخترنا port 8010 لأن port 8000 قد

5. طريقة تشغيل أسهل باستخدام Docker

الطريقة مفيدة لو Docker Desktop مثبت. ما زال يجب وضع مجلد models قبل build، لأن Dockerfile ينسخ النماذج داخل الصورة. هذه

```
git clone https://github.com/MohammedHssan11/f1-car-vision-intelligence-system.git  
cd f1-car-vision-intelligence-system  
  
copy .env.example .env  
# models/   
docker compose up --build
```

بعد انتهاء build افتح http://localhost:8010. لإيقاف الخدمة اضغط Ctrl+C ثم نفذ docker compose down عند الحاجة.

اختبار سريع بعد التثبيت

```
py -3.11 tests\test_reliability.py  
py -3.11 tests\smoke_phase1.py
```

ولذلك يأخذ وقتا أطول sample video والصورة و API يختبر smoke_phase1.py. سريع ولا يشغل فيديو كاملا test_reliability.py.

6. كيف يستخدم الواجهة؟

1. الصفحة الرئيسية /: اختصارات لصفحة الضرر وصفحة الفيديو والdemo actions.
2. /page/damage: يظهر app/static/results/ في annotated ثم ينشئ صورة upload التطبيق يحفظ. JPG/PNG/BMP ارفع download link.
3. /page/pipeline: background، لأن الفيديو يعمل في jobs الأفضل استخدام زر. tracker/tracker2.mp4 ارفع فيديو أو استخدم
job من صفحة progress ويمكن مراقبة
ويعمل في HTML5 player. لو فتحت نتيجة قديمة generated قبل آخر update، شغل job جديد ليتم ترميزه بالصيغة الجديدة.
4. بعد اكتمال job: افتح result page. الفيديو الجديد H.264/AAC

7. خريطة الملفات - الجذر والapp

README.md	ملخص المشروع، النماذج، الأداء، القيود، والfeatures.
requirements.txt	المطلوبة للتشغيل Python packages.

Dockerfile	يبنى Docker image ويثبت ffmpeg وPython packages.
docker-compose.yml	يشغل التطبيق على host port 8010 ويحفظ uploads/outputs على الجهاز.
.env.example	قيم environment variables الافتراضية التي يمكن نسخها إلى .env.
.gitignore / .gitattributes	يمنع رفع models والملفات generated files ويضبط التعامل مع الملفات الثنائية.
data.yaml	تعريف YOLO car dataset: class واحدة اسمها f1_car.
cleanup_outputs.py	حذف generated uploads/results بعد preview أو --yes.
app/main.py	ينشئ static mounts, CORS, FastAPI, ويضيف routes.
app/config.py	كل paths, حدود model hashes, uploads, وCORS settings.
app/api.py	كل endpoints, validation, حفظ templates rendering, uploads, وإدارة صفحات النتائج.
app/jobs.py	progress/status. لفديو واحد في كل مرة مع ThreadPoolExecutor background queue.

قوالب الواجهة داخل /app/templates

index.html	الصفحة الرئيسية.
damage_image.html	رفع صورة وعرض نتيجة damage detection.
full_pipeline.html	رفع/اختيار فيديو وتشغيل background job.
job_status.html	عرض progress الفيديو وإعادة التوجيه للنتيجة.
pipeline_result.html	download, وروابط, video player, statistics, النهائي debrief.

8. خريطة الملفات - الرؤية والتتبع

services/damage_image.py	src/detection.py يستدعي app/api.py بسيط يجعل wrapper
services/full_pipeline.py	src/pipeline.py يستدعي app/api.py و app/jobs.py بسيط يجعل wrapper
src/model_loader.py	مرة واحدة YOLO/FastAI وتحميل، GPU/CPU اختيار، hashes للنماذج، فحص lazy cache
src/detection.py	image damage detection، confidence filters، geometry rules لبعض damage classes.
src/pipeline.py	.frame loop، tracking، team classification، damage batching، annotations، H.264/AAC publish: المشروع
src/tracking.py	CarState: position، speed، acceleration، smoothed velocity، track history.
src/classification.py	model atomically ويكتب dataset-dir -- يأخذ JResNet34 team classifier training script
tracking_memory/tracker_core.py	قصيرة بحذر ID switches؛ يعالج ByteTrack بعد stable-ID resolver
tracking_memory/car_state.py	bridge لاستيراد CarState من src/tracking.py
tracking_memory/damage_state.py	يسجل persistence، observations، first/last seen، severity.
tracking_memory/damage_assigner.py	يربط damage box بالسيارة الصحيحة عبر $IoU > 0.3$.
tracking_memory/collision_detector.py	يصدر probable impact عند deceleration قوي و damages مؤكد زمنياً.
tracking_memory/overtake_detector.py	للسيارات النشطة فقط heuristic rank change
tracking_memory/utils.py	IoU helper.
tracker/bytetrack.yaml	إعدادات ByteTrack: thresholds، matching، buffer.
tracker/tracker2.mp4	فيديو demo موجود في repository للتشغيل السريع.
tracker/tracking_test.py	tracking محلي قديم لل visual test
tracker/tracking_test2.py	AVI يخرج فيديو headless tracking harness

9. خريطة الملفات - الاختبارات والbenchmarks والdocs

tests/test_reliability.py	لـ H.264 command، model integrity، ID reassociation، collision timing، Jinja rendering لل اختبارات سريعة
tests/smoke_phase1.py	health، upload validation، image path، و local video pipeline.
tests/smoke_phase2.py	demo buttons و rendering results.
tests/smoke_phase3.py	background job progress، failure state، result page.
tests/smoke_phase4.py	يتحقق من benchmark manifest و outputs.
benchmarks/manifest.json	قائمة samples المستخدمة في benchmark.
benchmarks/run_benchmarks.py	يشغل runtime models ويكتب results JSON/Markdown.
benchmarks/benchmark_results.md	نتائج benchmark الحالية وحدود labels المتاحة.
benchmarks/README.md	طريقة تشغيل benchmark وما الذي يقيسه.
docs/ARCHITECTURE.md	رسم architecture ومسار الطلب.

docs/AI_SYSTEM.md	النماذج، event logic، thresholds، وحدود التقييم.
docs/BACKEND.md	تفاصيل backend والأداء والـ pipeline.
docs/FRONTEND.md	شرح الصفحات والقوالب.
docs/API_REFERENCE.md	كل API routes وrequests/responses.
docs/DEPLOYMENT.md	تشغيل Windows/Docker وإعدادات البيئة.
docs/MAINTENANCE_GUIDE.md	اختبارات، outputs، تنظيف، tuning، retraining.
docs/SECURITY_AUDIT.md	مراجعة uploads، paths، CORS، model hashes.
docs/DATABASE.md	يوضح أنه لا توجد قاعدة بيانات حاليا.
docs/README.md / docs/full_doc.md	فهرس وأرشيف/شرح إضافي للوثائق.

source code ليست Folders

- models/: weights المطلوبة، ignored من Git.
- uploads/: ملفات المستخدم التي يرفعها.
- outputs/: الفيديوهات الناتجة.
- app/static/results/: الصور الناتجة.
- samples/: manifests ونتائج test samples الثقيلة؛ ignored.
- splitted_dataset/: benchmark ثلاث صور صغيرة للـ dataset موجودة، أما benchmark ثلاث صور صغيرة للـ dataset موجودة، أما benchmark ثلاث صور صغيرة للـ dataset موجودة، أما benchmark ثلاث صور صغيرة للـ dataset موجودة.

10. تسلسل تنفيذ الفيديو بالتفصيل

- 1. المستخدم يرفع فيديو أو يختار tracker/tracker2.mp4 من صفحة pipeline.
- 2. app/jobs.py في job والحجم وصلاحيه الفيديو ثم ينشئ extension يتحقق من app/api.py.
- 3. VideoJobStore تشغيل run_full_pipeline في worker داخل tracking state واحد لمنع تداخل.
- 4. src/model_loader.py فحص SHA-256 يحمل النماذج عند أول طلب فقط، بعد فحص.
- 5. لكل frame: YOLO يكتشف السيارة وByteTrack يعطي ID track raw.
- 6. TrackIdentityResolver يحتفظ stable ID لو حدث ID switch وغير ملتبس.
- 7. أول مرة تظهر فيها السيارة، classifier يتوقع team من crop السيارة.
- 8. كل damage model، DAMAGE_EVERY_N_FRAMES يفحص car crops batched ويربط النتيجة بالسيارة عبر IoU.
- 9. collision detector يجمع braking + damage confirmation. overtake detector يراقب rank change heuristic.
- 10. OpenCV يرسم compact labels. للمتصفح H.264/AAC MP4 +faststart يحول ffmpeg بعد النهاية.
- 11. job status يصبح completed، الرابط summary والواجهة تعرض، /outputs/videos/...

11. أهم environment variables

F1_APP_PORT	host port في Docker. default 8010.
ALLOWED_ORIGINS	CORS allow-list. default localhost:8010.
PIPELINE_CONF_TH	minimum confidence لاكتشاف السيارة. default 0.4.
PIPELINE_DAMAGE_CONF_TH	minimum confidence للضرر داخل الفيديو. default 0.4.
PIPELINE_TEAM_CONF_TH	minimum confidence لقبول team name. default 0.6.
DAMAGE_EVERY_N_FRAMES	كم frame بين damage inference. default 10.
MAX_FRAMES	حد أقصى للفريمات لتجنب فيديو طويل جدا. default 3000.
PIPELINE_H264_CRF	quality/size final video. أفضل يعني جودة أعلى وحجم أكبر. default 23.
PIPELINE_SHOW_SPEED	حتى لا تزدحم الشاشة speed labels. default false لإظهار true.
TRACK_REASSOCIATION_MAX_GAP	أقصى gap لتصحيح ID switch. default 30 frames.
COLLISION_MIN_DAMAGE_OBSERVATIONS	عدد damage observations اللازمة قبل impact event. default 2.

12. مشاكل شائعة وحلولها

Model not found	تأكد من وجود الملفات الثلاثة داخل /models بنفس الأسماء.
Integrity check failed	المرسل من صاحب المشروع أو حدث model ليس نفس النسخة المتوقعة. استخدم model file بعد مراجعة app/config.py في hash.
ffmpeg is required	ثبت ffmpeg أو استخدم Docker build، ثم تأكد من ffmpeg -version.

Video player black	تحديث. الناتج القديم mp4v قد لا يعمل في Chromium. أما الناتج الجديد H.264/AAC فيعمل. شغل job جديد بعد آخر
Port already in use	شغل على 8011 مثلاً: port 8011--، أو غير F1_APP_PORT عند Docker.
Raw {{ }} appears	لا تفتح app/templates/pipeline_result.html مباشرة. افتح URL من FastAPI فقط.
Slow processing	استخدم GPU CUDA، قلل MAX_FRAMES، أو ارفع DAMAGE_EVERY_N_FRAMES.
Too many labels	اترك PIPELINE_SHOW_SPEED=false. compact tags هي الوضع الافتراضي.

قبل أن تسلمه المشروع 13. Checklist

- أرسل رابط GitHub + مجلد models كاملاً.
 - اطلب منه أن يعمل git clone ثم ينسخ models / قبل تشغيل uvicorn.
 - اطلب منه تجربة api/health/ أولاً، ثم page/damage/، ثم page/pipeline/.
 - أرسل له هذا الـ PDF مع المشروع.
 - لو سيفتحه على جهاز مختلف تماماً، يفضل Docker لأن ffmpeg والتبعيات تنثبت داخل container.
- الذي يريد تفاصيل أعمق، يبدأ من README.md ثم docs/ARCHITECTURE.md ثم src/pipeline.py. انتهى الدليل. للمطور